TECH THINK²

Inteligência Artificial
além da nuvem

Introdução a Python

github.com/tech2think/python-learning

Vinicius Parede

CIENTISTA DE DADOS

🎓 **FORMAÇÃO**

Engenheiro Eletricista

Sistemas Eletrônicos Embarcados · UFPR

🔬 **PESQUISA**

Mestrando em Engenharia Elétrica

Linha de Pesquisa em Danos Elétricos · UFPR

💻 **ESPECIALIZAÇÃO**

Cientista de Dados · T2T

Python · Pandas · SQL · Snowflake · FastAPI · Docker · Git · IA Generativa · Agents · Claude Code

Agenda do Curso

Parte 1 — Python & Dados — 3h30

1. **Fundamentos Python**
   Variáveis · Tipos · Estruturas de controle · Funções
   1h15
2. **Análise de Dados com Pandas**
   DataFrames · Limpeza · Agrupamentos · Qualidade de dados
   1h30
3. **Visualização de Indicadores**
   matplotlib · seaborn · plotly · DEC / FEC
   45min

Parte 2 — IA & Snowflake — 3h30

4. **Claude Code & IA Generativa**
   Instalação · CLAUDE.md · Skills · Prompt engineering
   1h30
5. **Snowflake & Qualidade de Dados**
   Conexão PKI · Queries · Pipeline de qualidade de cadastro
   2h

Projeto prático: pipeline de qualidade do cadastro de consumidores integrado ao Snowflake da Equatorial

Total: 7 horas · 5 módulos práticos · Jupyter Notebooks + Apresentações

O que esperar deste curso?

OBJETIVO

Introduzir Python de forma prática para profissionais do **setor elétrico** — do básico até automação de análises e **qualidade de dados** integrada ao ambiente corporativo.

PROJETO PRÁTICO

Construção de um **Pipeline de Qualidade de Cadastro** integrado ao Snowflake, com validações automáticas e dashboards de indicadores.

PRÉ-REQUISITOS

- Entendimento de fluxos do setor elétrico
- Mentalidade voltada para automação de processos

FERRAMENTAS UTILIZADAS

Python 3.11 | Pandas | Snowflake | uv Manager | Claude Code

CÓDIGO FONTE:

github.com/tech2think/python-learning

Por que Python?

ANÁLISE DE DADOS

Processe milhões de registros de forma eficiente, identifique padrões e gere relatórios automáticos que levariam dias em planilhas convencionais.

INTEGRAÇÃO

Conecte-se facilmente a bancos de dados, sistemas ERP, APIs externas e diversas outras fontes de dados com poucas linhas de código.

AUTOMAÇÃO

Automatize validações de regras de negócio, rotinas de processamento diárias e o envio de alertas corporativos.

VISUALIZAÇÃO

Crie dashboards e gráficos interativos para monitorar indicadores de desempenho, facilitando a tomada de decisão gerencial.

INTELIGÊNCIA ARTIFICIAL

Implemente modelos preditivos, detecção de anomalias e integre suas aplicações facilmente com os modelos de linguagem mais modernos.

ECOSSISTEMA ABERTO

Linguagem gratuita, open source, suportada por uma comunidade global gigantesca e com bibliotecas prontas para quase qualquer necessidade.

Configuração do Ambiente com uv

Passo 1 — Instalar o uv

```
# Linux / Mac
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c \
  "irm https://astral.sh/uv/install.ps1 | iex"
```

uv é o gerenciador moderno do ecossistema Python — 10–100x mais rápido que pip, sem necessidade de instalar Python separadamente.

Opção B — Google Colab (Nuvem)

Caso tenha problemas com a instalação local ou restrições de TI:

Abrir Google Colab

Ambiente pronto para uso no navegador.
Requer apenas uma conta Google.

Basta fazer o upload dos notebooks do curso para o seu Drive.

Verificação: `uv run python --version` — deve mostrar `Python 3.11.x`

MÓDULO 1

Fundamentos Python

Construindo a base sólida necessária para automação e análise de dados.

M1 Variáveis e Tipos Básicos

Tipos Numéricos

```
# Inteiros (int) — contagens, códigos
cod_consumidor = 100042
tensao_nominal = 220 # Volts

# Ponto Flutuante (float) — medições, valores
consumo_kwh = 342.7
fator_potencia = 0.92
```

Textos e Lógicos

```
# Textos (str) — nomes, categorias
nome = "Maria Silva Santos"
municipio = "São Paulo"

# Booleanos (bool) — verdadeiro/falso
ativo = True
inadimplente = False
```

REGRAS IMPORTANTES

- Tipagem dinâmica: não precisa especificar o tipo na criação.
- Nomes não podem ter espaços ou caracteres especiais (exceto _).
- **Case Sensitive:** a variável X é diferente da variável x.
- Proibido usar palavras reservadas (`if`, `def`, `True`, etc.) como nomes.
- Use # para inserir comentários no seu código.

M1 Estruturas de Dados

Listas (Sequências)

Coleções ordenadas e mutáveis de itens.

```
# Criando uma lista de leituras
leituras = [310, 325, 342, 298, 315]

# Acessando o primeiro elemento (índice 0)
primeira_leitura = leituras[0]

# Adicionando um novo elemento
leituras.append(330)
```

Dicionários (Chave-Valor)

Coleções de pares com chaves únicas.

```
# Criando um registro de UC
consumidor = {
  "cod": 100042,
  "nome": "Maria Silva",
  "classe": "B1",
  "consumo_kwh": 342.7,
  "ativo": True
}

# Acessando valores
nome_cliente = consumidor["nome"]
```

REGRAS E DICAS

- **Listas:** o primeiro item está sempre no índice `0`.
- Use `len(lista)` para saber o número de itens.
- Listas preservam a ordem em que os itens foram inseridos.

- **Dicionários:** as chaves (ex: "nome") devem ser únicas.
- Acesso rápido: encontrar um valor pela chave é instantâneo.
- Ideal para representar objetos complexos (clientes, medidores).

M1 Estruturas de Controle

Classificação tarifária (if/elif)

```python
def classificar_tarifa(consumo_kwh):
  if consumo_kwh <= 30:
    return "Baixa Renda"
  elif consumo_kwh <= 100:
    return "Faixa 1"
  elif consumo_kwh <= 220:
    return "Faixa 2"
  else:
    return "Faixa 3"

print(classificar_tarifa(342))
# → "Faixa 3"
```

Iteração sobre leituras (for)

```python
leituras = [310, 325, 342, None, 315]

validas = []
for leitura in leituras:
  if leitura is not None:
    validas.append(leitura)

# List comprehension — mais pythonico
validas = [x for x in leituras
           if x is not None]

media = sum(validas) / len(validas)
print(f"Média: {media:.1f} kWh")
```

REGRAS E DICAS

- **Indentação é lei:** Em Python, os blocos de código são definidos pelos espaços no início da linha (use 4 espaços).
- **Não esqueça:** É obrigatório usar dois-pontos (`:`) no final das linhas de `if`, `elif`, `else` e `for`.
- **if/elif/else:** Em uma cadeia de condições, apenas o bloco da *primeira* condição verdadeira será executado.
- **for loop:** É a forma mais direta e pythônica de percorrer itens de uma lista um por um.

M1 Funções

Estrutura

```python
def nome_da_funcao(param1, param2):
  """Descrição do que a função faz."""
  # lógica aqui
  resultado = param1 + param2
  return resultado

# chamando a função
valor = nome_da_funcao(10, 20)
print(valor)  # 30
```

Exemplo — Cálculo do DEC

```python
def calcular_dec(interrupcoes: list) -> float:
  """Calcula o DEC em horas.
  Args: lista de dicts com
  'duracao_h' e 'consumidores'."""
  total_ucs = interrupcoes[0]['total_ucs']
  soma = sum(
    i['duracao_h'] * i['consumidores']
    for i in interrupcoes
  )
  return round(soma / total_ucs, 4)
```

REGRAS E DICAS

- **Reutilização:** Use funções para evitar repetir lógica de cálculo (como DEC/FEC) em vários lugares.
- **Docstrings:** Sempre documente o que a função faz, seus parâmetros e o que ela retorna entre `"""triple quotes"""`.
- **Type Hints:** Indicar tipos (ex: `cpf: str`) ajuda na leitura do código e evita bugs.
- **Modularidade:** Mantenha funções focadas em uma única tarefa para facilitar testes e manutenção.

MÓDULO 2

Análise de Dados com Pandas

Leitura, limpeza e agregação de dados.

M2 Pandas — Primeiros Passos

Lendo o cadastro de consumidores

```python
import pandas as pd

df = pd.read_csv("dados/consumidores_simulado.csv")

# Exploração básica
print(df.shape) # (50, 12)
df.head() # primeiras 5 linhas
df.info() # tipos e nulos
df.describe() # estatísticas

# Verificar nulos
df.isnull().sum()
```

Filtros e agrupamentos

```python
# Filtrar ativos
ativos = df[df['flg_ativo'] == True]

# Consumidores B1 com consumo > 300
altos = df[
  (df['cod_modalidade_tarifaria'] == 'B1')
  & (df['vlr_consumo_medio_kwh'] > 300)
]

# Agrupamento por classe
df.groupby('cod_classe_consumo')\
  ['vlr_consumo_medio_kwh']\
  .agg(['mean', 'count', 'sum'])\
  .round(2)
```

REGRAS E DICAS

- **DataFrame (df):** É a estrutura principal do Pandas, funcionando como uma tabela onde cada coluna possui um tipo de dado definido.
- **Exploração:** Sempre comece com `df.info()` para identificar colunas com valores nulos e tipos de dados errados.
- **Filtros:** Use parênteses `()` para separar condições múltiplas e o símbolo & para o "E" lógico.
- **Agrupamentos:** O `groupby` é o equivalente à Tabela Dinâmica, essencial para gerar indicadores por Classe ou UF.

M2 Limpeza de Dados de Consumidores

Problemas comuns no cadastro

- CPF/CNPJ com formatação inconsistente
- Nomes em maiúsculas, minúsculas e misto
- CEP sem zeros à esquerda
- Duplicatas de UC por migração de sistema
- Datas em formatos diferentes
- Campos obrigatórios vazios
- Coordenadas geográficas ausentes

Exemplos de Limpeza: Padronização de nomes, saneamento de CPF/CNPJ e remoção de registros duplicados em grandes bases cadastrais.

Técnicas de limpeza

```python
# Normalizar nomes
df['nom_consumidor'] = (
  df['nom_consumidor']
  .str.strip()
  .str.title()
)

# Limpar CPF/CNPJ
df['cpf_cnpj'] = (
  df['cpf_cnpj']
  .str.replace(r'\D', '', regex=True)
)

# Remover duplicatas
df = df.drop_duplicates(
  subset=['cod_consumidor']
)
```

REGRAS E DICAS

- **Padronização:** O uso de `.str.title()` garante que "MARIA", "maria" e "Maria" sejam tratadas como a mesma pessoa ("Maria").
- **Regex:** Expressões regulares (`r'\D'`) são poderosas para extrair apenas os números de CPFs mal formatados.
- **Duplicatas:** Ao remover duplicatas (`drop_duplicates`), defina qual coluna deve ser única com o parâmetro `subset`.
- **Impacto:** Lembre que dados de clientes limpos são cruciais para cobrança correta e relatórios regulatórios.

M2 Análise de Qualidade do Cadastro

Score de qualidade por registro

```python
def score_qualidade(row) -> float:
  campos = [
    'nom_consumidor', 'cpf_cnpj',
    'cod_modalidade_tarifaria',
    'nom_municipio', 'num_cep',
    'num_medidor', 'dat_ultima_leitura'
  ]
  preenchidos = sum(
    1 for c in campos
    if pd.notna(row[c]) and row[c] != ''
  )
  return round(preenchidos / len(campos), 2)

df['score_qualidade'] = df.apply(
  score_qualidade, axis=1
)
```

Dimensões de qualidade

```python
# Completude (%)
completude = (
  df['nom_consumidor'].notna().mean() * 100
)

# Unicidade (Duplicatas)
duplicados = (
  df.duplicated(subset=['cod_consumidor']).sum()
)

# Consistência
consistente = (
  df['cod_classe_consumo'].isin(['RES', 'IND', 'COM'])
)
```

REGRAS E DICAS

- **Completude:** Verifique se campos críticos (CPF, Medidor) estão preenchidos para evitar erros de faturamento.
- **Unicidade:** O `cod_consumidor` deve ser único para garantir a integridade da base.
- **Consistência:** Cruze dados (ex: Classe vs. Tarifa) para validar se as regras de negócio foram aplicadas.
- **Score:** Use o `.apply(axis=1)` para calcular indicadores linha a linha de forma simples.

MÓDULO 3

Visualização de Indicadores

Criação de gráficos estáticos, mapas de calor e dashboards interativos com Matplotlib, Seaborn e Plotly.

M3 Visualização de Indicadores

MATPLOTLIB — BARRAS

```
import matplotlib.pyplot as plt

classes = df.groupby(
  'cod_classe_consumo'
)['cod_consumidor'].count()

classes.plot(kind='bar',
  color='steelblue',
  title='Consumidores
  por Classe')
plt.tight_layout()
plt.show()
```

SEABORN — HEATMAP

```
import seaborn as sns

nulos = df.isnull().groupby(
  df['cod_uf']
).mean() * 100

sns.heatmap(
  nulos,
  annot=True,
  fmt='.0f',
  cmap='RdYlGn_r'
)
plt.title('% Nulos / UF')
```

PLOTLY — INTERATIVO

```
import plotly.express as px

fig = px.line(
  df_mensal,
  x='mes',
  y='consumo_kwh',
  color='classe',
  title='Consumo Mensal'
)
fig.update_layout(
  template='plotly_white'
)
fig.show()
```

REGRAS E DICAS

- **Matplotlib:** Ideal para gráficos estáticos rápidos e customizações minuciosas (ex: relatórios em PDF).
- **Seaborn:** Excelente para gráficos estatísticos complexos (como heatmaps de correlação) com poucas linhas de código.
- **Plotly:** A melhor escolha para dashboards interativos. Permite aos usuários dar zoom, filtrar e exportar dados.
- **Foco:** Escolha a ferramenta baseada no público. Relatórios impressos (matplotlib), exploração (seaborn), web (plotly).

M3 Indicadores DEC e FEC

DEC — DURAÇÃO EQUIVALENTE DE INTERRUPÇÃO

Soma das durações de interrupções ponderadas pelo número de UCs afetadas, dividida pelo total de UCs do conjunto.

```
# Fórmula
DEC = Σ(Duração_i × UCs_i) / UCs_total
```

FEC — FREQUÊNCIA EQUIVALENTE DE INTERRUPÇÃO

Número de interrupções ponderadas pelo número de UCs afetadas, dividido pelo total de UCs.

```
FEC = Σ(UCs_i) / UCs_total
```

```python
import plotly.graph_objects as go

fig = go.Figure()

fig.add_trace(go.Bar(
  name='DEC Real',
  x=meses, y=dec_real,
  marker_color='#dc2626'
))
fig.add_trace(go.Scatter(
  name='Limite ANEEL',
  x=meses, y=dec_limite,
  line=dict(color='#d97706',
    dash='dash')
))
fig.update_layout(
  title='DEC Mensal vs. Limite',
  template='plotly_white'
)
```

REGRAS E DICAS

- **Cálculo Crítico:** DEC e FEC são os indicadores de qualidade de fornecimento mais observados pela ANEEL e pela diretoria.
- **Ponderação:** Lembre que uma interrupção de 1h para 1000 clientes é mais grave no DEC do que 2h para 10 clientes.
- **Visualização:** Sobrepor o realizado (barra) com a meta regulatória (linha tracejada) é a forma padrão do setor de apresentar o DEC.
- **Qualidade:** Erros no cadastro de UCs (tema da aula anterior) afetam diretamente o denominador do cálculo!

Parte 1 Concluída!

Você concluiu com sucesso os fundamentos de Python, manipulação de dados com Pandas e visualização de indicadores.

Sua avaliação

Sua opinião é fundamental para melhorarmos o curso.

Acesse o formulário e responda:
"Qual seu grau de satisfação com a Parte 1 do curso?"

Escala de 1 (Muito Insatisfeito) a 5 (Muito Satisfeito).

forms.office.com/r/feedback